



UNIVERSIDAD DE LAS FUERZAS ARMADAS- ESPE - MATRIZ

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN - DCCO-SS

CARRERA DE INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN



ASIGNATURA

Metodología de la investigación

TEMA

Caja Blanca

INTEGRANTES

Jelen lucio, Jaime Cabezas, Bryan Cevallos

NIVEL-PARALELO - NRC

Tercero A – NRC : **30693**

FECHA DE ENTREGA

29/06/2026

SANGOLQUI – ECUADOR

REQUISITO FUNCIONAL N.º REQ002: Crear Documento

CÓDIGO FUENTE PARA CREAR DOCUMENTO

```
@app.post("/documentos")
async def crear_documento(
    request: Request,
    titulo: str = Form(...),
    idioma_origen: str = Form(...),
    idioma_destino: str = Form(...),
    fecha_entrega: str = Form(...),
    asignado_a: str = Form(...),
    comentarios: str = Form(""),
    archivo_origen: UploadFile = File(...),
    usuario_actual: dict = Depends(obtener_usuario_actual),
):
    if archivo_origen.content_type not in [
        "application/pdf",
        "application/msword",
        "application/vnd.openxmlformats-officedocument.wordprocessingml.document",
    ]:
        raise HTTPException(status_code=400, detail="Formato no permitido. Solo PDF o Word.")

    contenido = await archivo_origen.read()

    nombre_archivo = f"origen_{titulo.replace(' ', '_')}_{archivo_origen.filename}"
    ruta_origen = os.path.join("uploads", nombre_archivo)

    with open(ruta_origen, "wb") as buffer:
        buffer.write(contenido)
```

```

# Contar páginas para calcular el costo automáticamente

num_paginas = 1

if archivo_origen.content_type == "application/pdf":

    try:

        lector = PyPDF2.PdfReader(io.BytesIO(contenido))

        num_paginas = len(lector.pages)

    except Exception:

        num_paginas = 1

config = await request.app.state.db["configuracion"].find_one({"tipo": "costo"})

costo_por_pagina = config["costo_por_pagina"] if config else 5.00

costo = num_paginas * costo_por_pagina

nuevo_documento = {
    "titulo": titulo,
    "idioma_origen": idioma_origen,
    "idioma_destino": idioma_destino,
    "fecha_entrega": datetime.fromisoformat(fecha_entrega),
    "asignado_a": asignado_a,
    "comentarios": comentarios,
    "estado": "Pendiente",
    "fecha_envio": datetime.now(timezone.utc),
    "archivo_origen_url": ruta_origen,
    "archivo_traduccion_url": None,
    "creado_por": usuario_actual["correo"],
    "mensajes": [],
    "costo": costo,
}

```

```

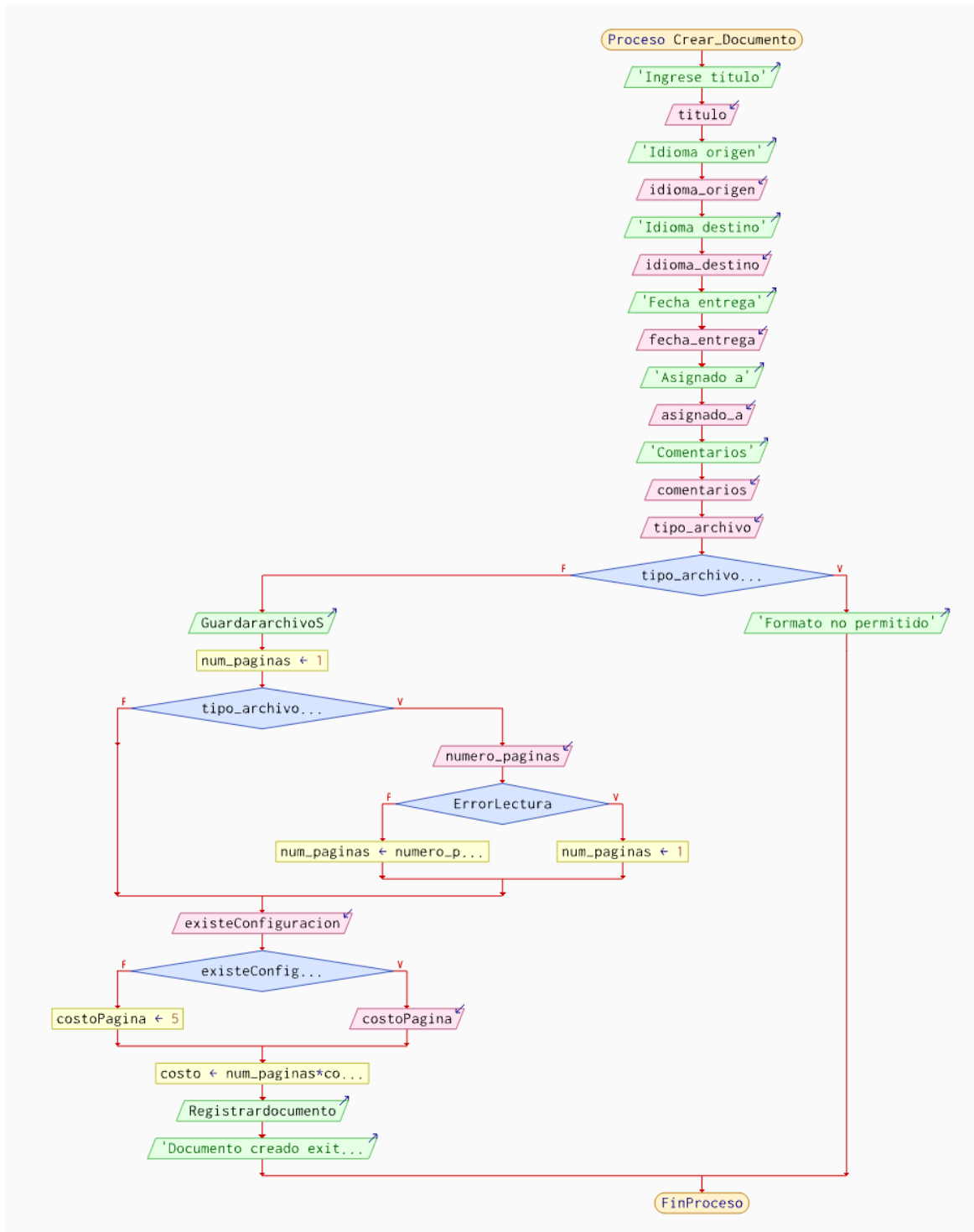
await request.app.state.db["documentos"].insert_one(nuevo_documento)

await registrar_log(usuario_actual["correo"], "Crear Documento", f"Título: {titulo}")

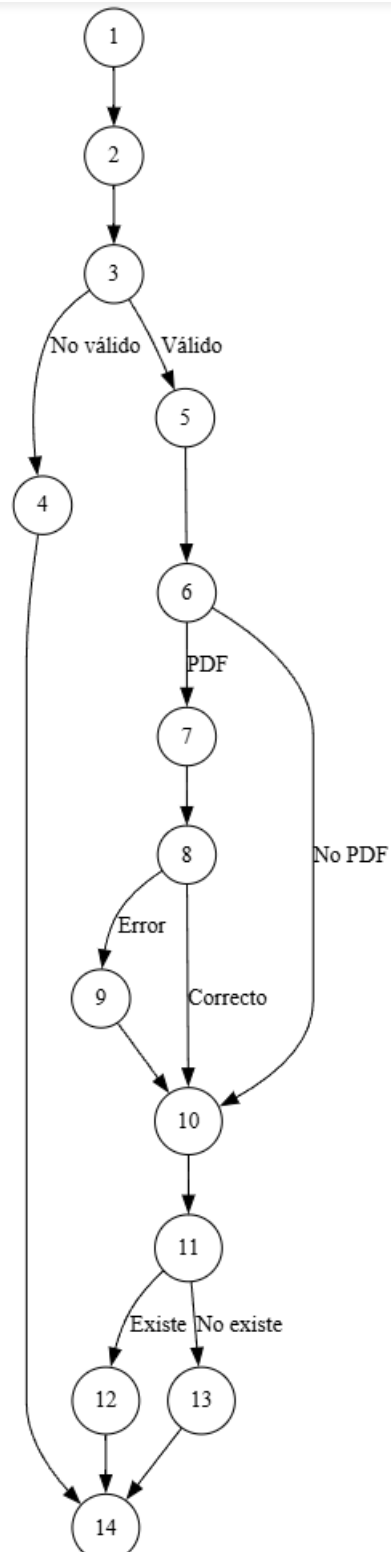
return {"mensaje": "Documento creado exitosamente"}

```

DIAGRAMA DE FLUJO (DF)



DIGRAFO



IDENTIFICACIÓN DE LAS RUTAS (Camino Básico)

R1: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 14$

(El archivo tiene un formato no permitido, se genera la excepción y termina el proceso.)

R2: $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9 \rightarrow 10 \rightarrow 11 \rightarrow 12 \rightarrow 14$

(El archivo es PDF, ocurre un error al leer el PDF, existe configuración de costo y el documento se registra correctamente.)

R3: $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 10 \rightarrow 11 \rightarrow 12 \rightarrow 14$

(El archivo es PDF, la lectura del PDF es correcta, existe configuración de costo y el documento se registra correctamente.)

R4: $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 10 \rightarrow 11 \rightarrow 12 \rightarrow 14$

(El archivo no es PDF, existe configuración de costo y el documento se registra correctamente.)

R5: $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 10 \rightarrow 11 \rightarrow 13 \rightarrow 14$

(El archivo no es PDF, no existe configuración de costo, se utiliza el valor por defecto y el documento se registra correctamente.)

COMPLEJIDAD CICLOMÁTICA

En el código de este requerimiento existen cinco nodos de decisión con condiciones que bifurcan el flujo lógico principal (Nodos predicados 3, 6, 8 y 11, además del manejo del error del PDF).

Los nodos predicados son:

Nodo 3

if archivo_origen.content_type not in [...]

Nodo 6

if archivo_origen.content_type == "application/pdf"

Nodo 8

try:

...

except Exception:

Nodo 11

if config

Por tanto:

$$P = 4$$

$$V(G)=P+1$$

$$V(G)=4+1$$

$$V(G)=5$$

2. Método por Aristas y Nodos

$$A = 17 \text{ aristas}$$

$$N = 14 \text{ nodos}$$

Sustituyendo en la fórmula de McCabe

$$V(G)=A-N+2$$

$$V(G)=17-14+2$$

$$V(G)=5$$

DONDE

P = Número de nodos predicado = 4

A = Número de aristas = 17

N = Número de nodos = 14

RESULTADO

$$V(G)=5$$

CONCLUSIÓN

La complejidad ciclomática del Requisito Funcional REQ002 – Crear Documento es 5, lo que indica que existen cinco caminos independientes de ejecución dentro de la lógica del proceso. En consecuencia, se requieren al menos cinco casos de prueba para verificar el correcto funcionamiento de la validación del formato del archivo, el procesamiento de documentos PDF, el manejo de errores durante la lectura del archivo, la obtención de la configuración del costo por página y el registro exitoso del documento en el sistema.